

Практическая работа №2

Знакомство с экосистемой Docker: от базовых команд к диагностике сервиса.

Персональные параметры (заполните ДО начала работы):

Параметр	Как определить	Ваше значение
Номер ИСУ	Можно взять из ЛК	—
Персональный порт	5430 + последние 2 цифры ису Пример: $5430 + 45 = 5475$	—
Персональный префикс	Фамилия латиницей в нижнем регистре (например, ivanov)	—
Секретный ключ	secret+последние 2 цифры ису+фамилия на английском	

!!!Важно!!! Все команды должны использовать ваши персональные значения. Отчёты с портом 5432 или без префикса <фамилия> — автоматически получают 0 баллов за соответствующие этапы.

Этап 1. Первый рабочий образ

Клонируйте репозиторий с приложением:
`git clone https://github.com/FilBTi/assist-py`

Задание 1.1 — Напишите базовый Dockerfile

Создайте файл Dockerfile в корне проекта. Он должен содержать:

```
# 1. Базовый образ (рекомендуем: python:3.11-slim)
FROM _____

# 2. Метаданные — ОБЯЗАТЕЛЬНО укажите себя как мейнтейнера!
LABEL maintainer="<ваша_фамилия>@lab.local"
LABEL version="1.0"

# 3. Рабочая директория
WORKDIR _____

...EXPOSE _____

# 8. Запуск приложения с вашим портом
CMD _____
```

Подсказки:

Базовый образ: `python:3.11-slim`

`WORKDIR` → `/app`

Сначала копируем `requirements.txt`, потом `app/`

Команда запуска: `uvicorn app.main:app --host 0.0.0.0 --port <ваш_порт>`

Задание 1.2 — Соберите образ

```
docker build -t <фамилия>-assistant:v1 .
```

Задание 1.3 — Запустите контейнер с переменными окружения

```
docker run -d \
```

```
--name <фамилия>-assistant-1 \  
-p <ваш_порт>:<ваш_порт> \  
-e APP_OWNER=<ваша_фамилия> \  
-e APP_SECRET=<ваш_секретный_ключ> \  
<фамилия>-assistant:v1
```

Задание 1.4 — Проверьте работоспособность

```
# Проверка главной страницы  
curl http://localhost:<ваш_порт>/  
  
# Проверка персонализации  
curl http://localhost:<ваш_порт>/me  
  
# Тест заметок  
curl -X POST http://localhost:<ваш_порт>/notes \  
-H "Content-Type: application/json" \  
-d '{"text": "ЛР2: сборка образа"}'  
  
curl http://localhost:<ваш_порт>/notes  
  
# Тест секретного эндпоинта (должен вернуть флаг!)  
curl http://localhost:<ваш_порт>/secret/<ваш_секретный_ключ>
```

Требование для отчёта, скриншот терминала с выводом всех 4 команд curl.
На скриншоте должна быть видна ваша фамилия в ответе /me и флаг в ответе /secret.

Этап 2. Оптимизация образа

Задание 2.1 — Улучшите Dockerfile

Создайте новый файл `Dockerfile.optimized` со следующими улучшениями: `requirements.txt` копируется отдельно от кода (уже сделано в Этапе 1 — убедитесь, что так и есть!). Очистка кеша `pip`. Multi-stage сборка (бонус): необязательно для базового задания, но даёт +1 балл

Пример улучшенной установки:

```
RUN pip install --no-cache-dir -r requirements.txt
```

Задание 2.2 — Сравните размеры

```
# Собираем оптимизированный образ
docker build -t <фамилия>-assistant:v2 -f Dockerfile.optimized .

# Сравниваем размеры
docker images | grep <фамилия>-assistant
```

На сколько МБ уменьшился образ? Почему копирование `requirements.txt` отдельно от кода ускоряет повторные сборки при изменении бизнес-логики?

В отчет нужно добавить скриншот вывода `docker images` с двумя версиями образа (`v1` и `v2`).

Этап 3. .dockerignore — чистота в образе

Задание 3.1 — Создайте .dockerignore

В корне проекта создайте файл .dockerignore со следующим содержимым:

```
__pycache__/  
*.pyc  
*.pyo  
.git  
.gitignore  
.venv/  
venv/  
.env  
*.log  
Dockerfile*  
README.md
```

Задание 3.2 — Пересоберите образ

```
docker build -t <фамилия>-assistant:v3 .
```

Какие файлы могли попасть в образ без .dockerignore? Почему наличие .git в образе — потенциальная уязвимость безопасности?

Этап 4. Healthcheck — готовность к оркестрации

Задание 4.1 — Добавьте проверку здоровья

В конец Dockerfile.optimized (после EXPOSE) добавьте:

```
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
  CMD curl -f http://localhost:<ваш_порт>/health || exit 1
```

Пояснение флагов:

--interval=30s — проверять каждые 30 секунд

--timeout=5s — ждать ответа не дольше 5 секунд

--start-period=10s — дать контейнеру 10 секунд на «прогрев» после старта

--retries=3 — считать сервис нездоровым после 3 неудачных проверок

Задание 4.2 — Соберите финальную версию

```
docker build -t <фамилия>-assistant:final -f Dockerfile.optimized .
```

Задание 4.3 — Проверьте статус здоровья

```
# Запускаем контейнер
docker run -d \
  --name <фамилия>-assistant-final \
  -p <ваш_порт>:<ваш_порт> \
  -e APP_OWNER=<ваша_фамилия> \
  -e APP_SECRET=<ваш_секретный_ключ> \
  <фамилия>-assistant:final

# Ждём инициализацию (10 секунд)
sleep 10

# Проверяем статус — должен быть (healthy)
docker ps --filter "name=<фамилия>-assistant-final" --format "table
{{.Names}}\t{{.Status}}"
```

В отчет добавьте скриншот вывода docker ps, где чётко виден статус (healthy).

Этап 5. Финальный артефакт и тегирование

Задание 5.1 — Добавьте семантические теги

```
# Тег с версией
docker tag <фамилия>-assistant:final <фамилия>-assistant:1.0.0

# Тег с датой сборки
docker tag <фамилия>-assistant:final <фамилия>-assistant:$(date
+%Y%m%d)
```

Задание 5.2 — Артефакт выполнения (защита от копирования!)

```
echo "===== " && \
echo "Сборка завершена: $(date '+%d.%m.%Y %H:%M:%S') " && \
echo "Студент: $(whoami)@$(hostname)" && \
echo "Образ: <фамилия>-assistant" && \
echo "===== " && \
docker images | grep <фамилия>-assistant | awk '{print $1, $2, $7}'
```

Скриншот полного вывода этой команды. На скриншоте должны быть видны:

1. Ваш логин и хост (whoami@hostname)
2. Текущее время выполнения
3. Все теги образа (final, 1.0.0, дата)
4. Размеры образов

Требования к отчёту

Файл должен называться: ПР2_Фамилия_ИСУ.pdf (например, ПР1_Иванов_4123.pdf)

Обязательные разделы:

1. Заполненная таблица персональных параметров
2. Скриншоты
3. Полный текст вашего финального Dockerfile.optimized (с комментариями)
4. Содержимое .dockerignore